

KNOWLEDGE TRANSFER / APPLICATION DEVELOPMENT

project handover

HMC Sanaad B2E Platform

Project Handover & Developer Onboarding
HMC_BackEnd (Sanaad API) · HMC_Gateway (Public Gateway)
NestJS 11 · Node.js 20 · Oracle · SQL Server



HMC / EMPLOYEE SELF-SERVICE PLATFORM

CONTENT VERSION

1.0 / 30 August 2026

SCOPE

02 applications

HANDOVER

21 chapters

NAVIGATE THE HANDOVER

inside this handover

Audience: a developer who is new to this project and has little or no experience with Node.js, TypeScript, or NestJS.

Goal: after reading this document you should be able to run, understand, debug, maintain, and extend the system with minimal help from the previous developer.

Source snapshot: version 1.0, dated 2026-08-30. This revised edition applies the requested content scope and standard command notation; it does not revalidate the remaining technical guidance. Verify current implementation against the repositories and AGENTS.md.

01	Project Overview	03	12	Configuration & Environment Variables	27
02	Technology Stack	05	13	API Request Lifecycle	30
03	Environment Setup	06	14	Error Handling	31
04	Required Node.js + TypeScript Basics	09	15	How to Develop a New Feature	32
05	NestJS Basics for This Project	11	16	Testing & Debugging	34
06	Project Structure	13	17	Important npm Commands	35
07	Application Startup Flow	15	18	Troubleshooting	36
08	Module-by-Module Documentation	16	19	Learning Roadmap (First Week)	37
09	Authentication & Authorization	22	20	Learning Resources	38
10	Database Architecture	24	21	Requires Knowledge Transfer	39
11	External Integrations	26			

Start locally / Chapters 1–7 establish the platform, tools, setup, and application structure.

Understand operations / Chapters 8–14 cover modules, authentication, data, configuration, and error handling.

Take ownership / Chapters 15–21 cover delivery, testing, troubleshooting, learning, and team knowledge transfer.

CHAPTER 01 / 21

project overview

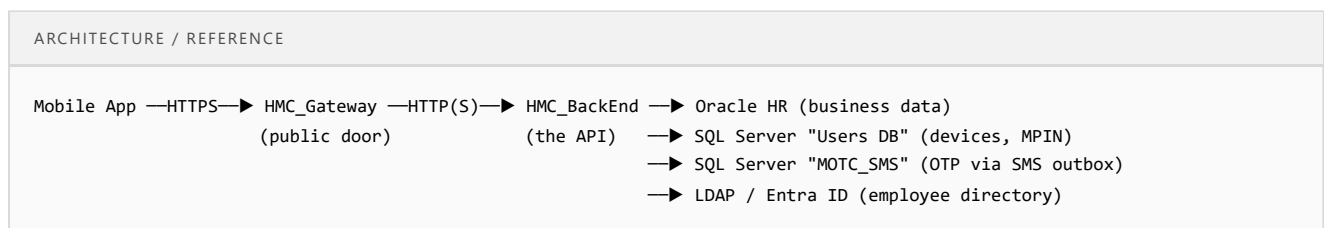
Project in 5 Minutes

Sanaad is HMC's (Hamad Medical Corporation) employee self-service mobile app. Employees use it to see their payslips, request leave, update their personal details, request HR letters, manage dependents, approve requests from their team, and more.

All the *real* business logic and data already exist in HMC's **Oracle HR system** (as database views and PL/SQL procedures named `XXHMC_SND_*`) and in a legacy **SQL Server** database (login devices, MPINs, OTPs). This project does **not** re-implement that logic. Instead, it is a modern, secure **API layer** that:

1. Lets the mobile app log in safely (OTP by SMS + a 4-6 digit MPIN → a JWT token),
2. Translates clean REST API calls (e.g. `GET /api/v1/leave/balance`) into the correct Oracle view reads and procedure calls,
3. Returns results in the exact JSON shape the mobile app expects (including Arabic/English localization).

There are **two applications**:



- **HMC_Gateway** is the only thing exposed to the internet. It forwards login-journey requests to the backend, checks the JWT on every other request (without calling the backend), applies rate limiting against brute-force attacks, and proxies everything else through untouched.
- **HMC_BackEnd** is the actual API: feature modules re-exposing legacy Sanaad operations.

Main users: HMC employees (mobile app).

Key design idea to remember: *the database is the source of truth for business logic; this codebase is a disciplined translator.* Most bugs are solved by looking at what Oracle/SQL Server actually expects.

Main Features (by module)

Area	What the employee can do
Auth	Log in with OTP + MPIN, refresh session, logout, forgot MPIN
Profile / Employee	View & update personal details, view employment/salary history, change supervisor

Area	What the employee can do
Payslip	List pay periods, generate a full payslip
Leave	Check balance, apply/amend/cancel leave, return from leave, leave history
Letters	Request HR letters/certificates
Identity	View/update QID, request company ID card
Contact	Update phones and addresses
Dependents	Add/update/delete dependents, passport details
School Fees	Apply for children's school-fee support
Annual Ticket	Apply for / cancel the annual flight ticket benefit
Approvals	Managers approve/reject/reassign requests, worklist

High-Level Architecture

Every backend feature module follows the same **Clean Architecture** layering:

ARCHITECTURE / REFERENCE	
interface/	← Controller + DTOs (HTTP shape, validation)
application/	← Service (orchestration, no SQL here)
domain/	← Repository interface ("port") + domain types – framework-free
infrastructure/	← Oracle/SQL/HTTP adapter implementing the port

Cross-cutting concerns (config, database pools, JWT auth, error handling, response envelope) live once in `src/core/` and apply to every request automatically.

CHAPTER 02 / 21

technology stack

Technology	Version	Purpose	Used In
Node.js	20.x (Docker uses node:20-slim)	JavaScript runtime	Both projects
TypeScript	5.x	Typed language, compiled to JS	Both projects
NestJS	^11.1.17	Application framework (modules, DI, HTTP)	Both projects
Express	(via @nestjs/platform-express)	Underlying HTTP server	Both projects
node-oracledb	^6.5.0	Oracle database driver (connection pool)	Backend
mssql (tedious)	^11.0.1	SQL Server driver — two pools (Users DB, MOTC SMS DB)	Backend
@nestjs/jwt + passport-jwt	^11.0.2 / ^4.0.1	Issue & verify JWT tokens	Both (backend issues, both verify)
Idapts	^9.0.0	LDAPS directory lookups	Backend
@nestjs/axios + axios	^4.0.1 / ^1.6.8	Outbound HTTP (Entra Graph, SMS) & proxying	Both
@nestjs/config + Joi	^4.0.4 / ^17.13.0	Typed configuration + env validation at boot	Both
class-validator / class-transformer	^0.14.1 / ^0.5.1	Request DTO validation	Both
@nestjs/throttler	^6.4.0	Rate limiting on login endpoints	Gateway
helmet / compression	^8.1.0 / ^1.7.4	Security headers / gzip	Both
Jest + ts-jest + Supertest	^29.x / ^7.0.0	Unit & e2e testing	Both
Docker (multi-stage)	node:20-slim + Oracle Instant Client	Container deployment	Both

CHAPTER 03 / 21

environment setup

3.1 Prerequisites

1. **Node.js 20 LTS** — install from <https://nodejs.org> (check with `node -v`).
2. **Git**.
3. **Visual Studio Code** (recommended) with the ESLint and Prettier extensions.
4. Optional for full local runs: access to the HMC network (Oracle/SQL Server/LDAP are internal). **You can run everything without any database** — see 3.5.

3.2 Get the code and install

BASH / REFERENCE

```
git clone <repository-url>          # ← ask the team for the actual repo URL(s)
cd development/HMC_BackEnd
npm ci                               # install exact locked dependencies

cd ../HMC_Gateway
npm ci
```

`npm ci` reads `package-lock.json` and installs the exact dependency tree (preferred over `npm install` for reproducibility).

3.3 Configure environment variables

Each project reads a `.env` file at its root. Start from the documented template:

BASH / REFERENCE

```
# in HMC_BackEnd/
copy .env.example .env
# in HMC_Gateway/
copy .env.example .env
```

Open each `.env` and fill in what you have. Every variable is explained inside `.env.example` and in [Section 12](#). **Never commit** `.env`.

3.4 Build and start

BASH / REFERENCE

```
# Backend (from HMC_BackEnd/)
npm run build          # compile TypeScript → dist/
npm run start:dev      # OR: hot-reload development server

# Gateway (from HMC_Gateway/)
npm run build
npm run start:dev
```

3.5 Run WITHOUT any databases (the standard local smoke test)

The backend boots happily with every external system disabled — dependency-injection still wires up and routes register. This is how you verify your environment quickly:

POWERSHELL / REFERENCE

```
# Terminal 1 – HMC_BackEnd
cd HMC_BackEnd
$env:ORACLE_DISABLED='true'; $env:AUTH_DISABLED='true'; $env:LDAP_ENABLED='false'
$env:JWT_SECRET='local_dev_secret_local_dev_secret_1234'; $env:PORT='3009'
node dist/main.js

# Terminal 2 – HMC_Gateway
cd HMC_Gateway
$env:BACKEND_BASE_URL='http://localhost:3009'; $env:BACKEND_API_PREFIX='api/v1'
$env:JWT_SECRET='local_dev_secret_local_dev_secret_1234'; $env:AUTH_DISABLED='true'
$env:GATEWAY_PORT='3001'
node dist/main.js
```

3.6 Verify it works

In Postman, send `POST http://localhost:3001/api/v1/auth/login` with a JSON body containing `username`, `mpin`, `imeinumber`, `platform`, and `version`. Expect a signed JWT (dev bypass), confirming the full gateway → backend proxy path.

3.7 Docker (optional)

Both projects ship a multi-stage `Dockerfile` (backend includes the Oracle Instant Client) and a `docker-compose.yml`:

BASH / REFERENCE

```
cd HMC_BackEnd
docker compose up --build
```

Compose loads `.env` via `env_file` and additionally maps a specific list of host variables (Oracle/LDAP/Users-DB/SMS) in its `environment:` block — host values win over `.env`. Caution: Newer variables (`MOTC_SMS_*`, `OTP_STORE`, `AUTH_STATIC_LOGIN`) are **not** in that block yet, so in Docker they must be set in `.env` (see Section 21).

DNS inside containers: short hostnames like `HPDSLDEVV01` do **not** resolve inside Linux containers. Use the FQDN (e.g. `HPDSLDEVV01.hmc.org.qa`) or an IP, or add an `extra_hosts` entry.

CHAPTER 04 / 21

required node.js + typescript basics

Only what you need for *this* project.

npm & package.json

- **What:** npm is Node's package manager; `package.json` declares dependencies and runnable scripts.
- **Why:** everything you run goes through it (`npm run build`, `npm test`).
- **Here:** each project has its own `package.json`. The `scripts` section is your command menu (Section 17). `package-lock.json` pins exact versions — commit it, don't edit it by hand.

async/await & Promises

- **What:** a `Promise` represents a value that arrives later (e.g. a database answer). `await` pauses the function until it arrives; such functions are marked `async`.
- **Why:** every database/HTTP call in this codebase is asynchronous.
- **Here:** e.g. `HMC_BackEnd/src/modules/leave/infrastructure/oracle/leave.oracle.repository.ts` — `async` `getBalance(...)` awaits an Oracle call. Fan-out reads use `Promise.all([...])` to run several reads in parallel (e.g. `profile.oracle.repository.ts`).

Environment variables

- **What:** key=value settings read from the process environment (or a `.env` file) at startup.
- **Why:** the same code runs in dev/staging/production with different databases and secrets.
- **Here:** loaded by `@nestjs/config` from `.env`, validated by `Joi` (`src/core/config/env.validation.ts`), and exposed as typed objects (`src/core/config/configuration.ts`). Code never reads `process.env` directly outside that file.

TypeScript basics

- **What:** JavaScript + types. Compiled to plain JS in `dist/` by `npm run build`.
- **Why:** types catch mistakes at build time; the whole codebase relies on them.

- **Key concepts used here:**
 - **Interfaces** — describe object shapes, e.g. `LeaveBalanceQuery` in `src/modules/leave/domain/leave.repository.ts`; repository "ports" are interfaces (`OtpPort`, `LovRepository`).
 - **Classes** — controllers/services/DTOs are classes because NestJS decorators attach to them.
 - **Decorators** — `@Controller()`, `@Get()`, `@Injectable()`, `@IsString()` ... metadata annotations NestJS/class-validator read at runtime.
 - **Generics** — `query<T>(…): Promise<T[]>` lets a helper return typed rows.
 - **Path aliases** — imports like `@core/…`, `@shared/…` map to `src/core`, `src/shared` (see `tsconfig.json` → `paths`).
- **Modules (ES/TS)** — every file exports things (`export class …`) and imports them; not to be confused with *NestJS modules* (next section).

CHAPTER 05 / 21

nestjs basics for this project

NestJS organizes an app into **modules** containing **controllers** (HTTP layer) and **providers/services** (logic), glued together by **dependency injection**.

Modules (`@Module`)

A module groups related pieces and declares what it imports/provides/exports.

- Root: `src/app.module.ts` imports `CoreModule` + all feature modules.
- Example: `src/modules/leave/leave.module.ts` declares the leave controllers, service, and binds the repository interface to its Oracle implementation.

Controllers (`@Controller`, `@Get`, `@Post`)

Receive HTTP requests, validate input via DTOs, call a service, return the result.

- Example: `src/modules/leave/interface/leave.controller.ts` — `@Controller('leave')` + `@Get('balance')` handles `GET /api/v1/leave/balance`.
- Controllers contain **no business logic and no SQL**.

Services / Providers (`@Injectable`)

Hold orchestration logic; injected into controllers via constructors.

- Example: `src/modules/leave/application/leave.service.ts`.

Dependency Injection (DI)

You never write `new LeaveService()`. You declare constructor parameters and Nest supplies instances. Interfaces are bound via **tokens**:

TS / REFERENCE

```
// leave.module.ts
{ provide: LEAVE_REPOSITORY, useClass: LeaveOracleRepository }
// leave.service.ts
constructor(@Inject(LEAVE_REPOSITORY) private readonly repo: LeaveRepository) {}
```

This is why swapping an implementation (e.g. OTP store `motc` vs `legacy` in `src/modules/auth/auth.module.ts`) is a one-line factory, not a rewrite.

DTOs + ValidationPipe

DTO = a class describing a request's shape with validation decorators:

```
TS / REFERENCE

export class LeaveBalanceQueryDto extends LangQueryDto {
  @IsString() @NotEmpty() username?: string;
}
```

A **global ValidationPipe** (`src/core/core.module.ts`) runs with `whitelist: true`, `forbidNonWhitelisted: true` — unknown JSON keys are **rejected with 400**, and declared fields are validated/transformed automatically.

Guards

Run before the handler and decide "may this request proceed?".

- `JwtAuthGuard` (global) — requires a valid bearer token unless the route is `@Public()`.
- `RolesGuard` — enforces `@Roles(...)`.
- `FunctionAccessGuard` — enforces `@RequireFunction('CODE')` against the `functions` claim in the JWT. All in `src/core/auth/`.

Interceptors

Wrap around the handler (before + after). Global interceptors covered here: `TimeoutInterceptor` (408 after `REQUEST_TIMEOUT_MS`) → `ResponseInterceptor` (wraps results in the Sanaad envelope + Arabic/English localization; `@SkipEnvelope()` opts out — the auth journey uses it).

Exception Filters

`AllExceptionsFilter` (`src/core/http/all-exceptions.filter.ts`) catches **every** error, classifies it (validation / DB / external service / auth...), and returns a safe, consistent JSON error to the client.

Decorators cheat-sheet (project-specific)

Decorator	Meaning	Defined in
<code>@Public()</code>	Skip JWT guard	<code>core/auth/decorators/public.decorator.ts</code>
<code>@CurrentUser()</code>	Inject the authenticated user	<code>core/auth/decorators/current-user.decorator.ts</code>
<code>@Roles(...)</code> / <code>@RequireFunction('code')</code>	Authorization	<code>core/auth/decorators/</code>
<code>@SkipEnvelope()</code>	Return raw JSON (no Sanaad envelope)	<code>core/http/response.interceptor.ts</code>

CHAPTER 06 / 21

project structure

6.1 Workspace

ARCHITECTURE / REFERENCE

```
development/
├─ AGENTS.md          ← KEY: operational knowledge base: verified DB facts, build quirks, staging findings. READ IT.
├─ Docs Project/      ← client-provided legacy service mapping (sanaad-api-service-mapping.html = proc-parameter
truth)
├─ Docs_Ai/           ← architecture & domain documentation referenced by the README
├─ PROJECT_ANALYSIS.md ← the technical analysis behind this document
├─ HMC_BackEnd/       ← the Sanaad API
└─ HMC_Gateway/       ← the public gateway
```

6.2 HMC_BackEnd/src

ARCHITECTURE / REFERENCE

```
src/
├─ main.ts            ← bootstrap: 15mb body limit, helmet, compression, CORS, /api/v1 prefix
├─ app.module.ts      ← imports CoreModule, LookupsModule + feature modules
├─
├─ core/              ← cross-cutting, applies to every request
│ └─ core.module.ts   ← global pipe/interceptors/guards/filter registration (ORDER MATTERS – see §13)
│ └─ config/          ← configuration.ts (15 typed namespaces) + env.validation.ts (Joi, fails boot on bad env)
│ └─ database/
│ │ └─ oracle.service.ts      ← THE Oracle pool: query/call/callCursor/callMultiCursor
│ │ └─ oracle-schema.service.ts ← asks Oracle's data dictionary for real column/parameter names (never guess!)
│ │ └─ base.repository.ts     ← base class every Oracle adapter extends (readByResolvedKey, callSubmitProc,
callRowsProc...)
│ │ └─ mssql.service.ts       ← SQL Server pool #1: Users/Sanaad DB (devices, MPIN, legacy OTP, app tables)
│ │ └─ motc-sms-db.service.ts ← SQL Server pool #2: MOTC_SMS DB (OTP push table = SMS delivery)
│ └─ auth/                  ← JwtStrategy, JwtAuthGuard, RolesGuard, FunctionAccessGuard, TokenRevocationService,
decorators
│ └─ http/                  ← ResponseInterceptor (envelope+localization), TimeoutInterceptor, AllExceptionsFilter,
exception classifier
│
├─ shared/                  ← reusable, dependency-free building blocks
│ └─ constants/oracle-objects.ts ← KEY: allow-list of ~95 XXHMC_SND_* names – ONLY these can be queried
│ └─ constants/oracle-columns.ts ← key-column candidate lists for view filtering
│ └─ constants/lov-names.ts     ← public lovname/lookupname → Oracle object registry
│ └─ constants/error-codes.ts   ← ORA-code extraction & HTTP mapping
│ └─ domain/ (lang.ts, lov-item.ts, submit-result.ts)
│ └─ dto/ (lang-query, common-query, oracle-submit helpers, pagination, envelopes)
│ └─ utils/ (date.util, localize.util KEY: AR/EN twin collapsing, mapper.util, blob.util, url-decode.util)
├─ lookups/                  ← shared-kernel LOV service (cached Oracle view reads) – used by ALL feature modules
│ └─ interface/lookups.controller.ts (/lookups/yes-no, /rfmi-user, /lov, /master)
│ └─ application/lookups.service.ts
│ └─ infrastructure/oracle/lov.oracle.repository.ts (cache, request coalescing, dictionary-resolved filters)
```

ARCHITECTURE / REFERENCE / CONTINUED

```
└─ modules/                ← feature modules, all with the same internal layout:
  │   └─ auth/              ← interface/ (controller+DTOs) · application/ (services) · domain/ (ports) · infrastructure/
  │   (adapters)
  │   │   └─ profile/ employee/ payslip/ leave/ letters/ identity/
  │   │   └─ contact/ dependents/ school-fees/ annual-ticket/ approvals/
```

6.3 HMC_Gateway/src

ARCHITECTURE / REFERENCE

```
src/
└─ main.ts                  ← helmet, compression, CORS, /api/v1 prefix
└─ app.module.ts           ← CoreModule → AuthModule → ProxyModule (wildcard imported LAST)
└─ core/
  │   └─ core.module.ts     ← config+Joi, ThrottlerModule, global ValidationPipe + JwtAuthGuard + AllExceptionsFilter
  │   └─ auth/              ← local JWT validation (shared secret with backend) + @Public/@CurrentUser
  │   └─ config/            ← configuration.ts + env.validation.ts
  │   └─ http/              ← shared axios client with backend TLS agent; minimal error filter
└─ modules/
  │   └─ auth/              ← forwarded @Public journey routes (login/OTP/MPIN/refresh – throttled) + /healthcheck
  │   └─ proxy/             ← @All('*') wildcard → ProxyService (byte-for-byte relay; 502/504 only on network failure)
```

CHAPTER 07 / 21

application startup flow

Backend (HMC_BackEnd)

ARCHITECTURE / REFERENCE

```
node dist/main.js
↓
main.ts bootstrap()
├─ NestFactory.create(AppModule)           ← builds the DI container
│   ↓
│   CoreModule loads .env → Joi validation ← BAD ENV = APP REFUSES TO START (fail fast)
│   ↓
│   OracleModule / MssqlModule onModuleInit():
│   │   └─ Oracle pool created (skipped if ORACLE_DISABLED=true)
│   │   └─ Users DB pool created (skipped if USERS_DB_DISABLED=true or host missing)
│   │   └─ MOTC SMS DB pool created (skipped if MOTC_SMS_DB_DISABLED=true or host missing)
│   │       Caution: a pool that has credentials but CANNOT connect THROWS → boot fails (by design)
├─ body-parser limits set to 15mb (base64 attachments in submits)
├─ helmet(), compression(), CORS
├─ setGlobalPrefix('api/v1')
└─ app.listen(PORT)
```

Gateway (HMC_Gateway)

Same shape, no databases: config validation → shared axios client (with optional CA cert for an https backend) → guards/filters → listen on `GATEWAY_PORT`. Route registration order guarantees the literal `/auth/*`, `/healthcheck` routes win over the `@All('*')` wildcard.

CHAPTER 08 / 21

module-by-module documentation

The endpoints below cover the feature modules included in this handover. Every route is prefixed with `/api/v1`. **Auth column:** Public = @Public (no token), JWT = bearer JWT required. The general flow for every module is: Client → Controller (DTO validation) → Service → Repository port → Oracle/SQL adapter → mapped response → Sanaad envelope.

8.1 auth — the Sanaad authentication journey (+ sessions)

Location: `src/modules/auth` · **External systems:** Users DB (SQL Server), MOTC SMS DB, LDAP/Entra directory. No Oracle.

Method	Endpoint	Description	Auth
POST	/healthcheck	API-1 — app-launch check (downtime + forced update)	Public
POST	/auth/initiate	API-2 — validate user, resolve phone, send OTP	Public
POST	/auth/otp/validate	API-3 — validate OTP	Public
POST	/auth/mpin/update	API-4 — set MPIN (first time)	Public
POST	/auth/login	API-5 — MPIN login → JWT + refresh token + functionaccesslist	Public
POST	/auth/mpin/forgot	API-6 — forgot MPIN (send OTP)	Public
POST	/auth/mpin/update/reset	API-7 — reset MPIN with OTP	Public
POST	/auth/token/refresh	exchange refresh token for a new pair (rotated, one-time use)	Public
POST	/auth/logout	revoke current access token (+ optional refresh token)	JWT
GET	/auth/me	current identity from the JWT	JWT

Key implementation facts:

- Everything is a **port** with swappable adapters (see §9 and §11): directory (`AUTH_DIRECTORY`), OTP store (`OTP_STORE`), MPIN/device store, function-access source.
- OTP (default `motc` store): generated in code, INSERTed into `MOTC_SMS_PushTable` — **the insert IS the SMS send** (government gateway polls the table). Validation reads the same table back. `MessageID` doubles as the mobile `requestid`.

- MPIN is stored **as received** (the client pre-hashes) and compared by SQL equality — a deliberate legacy-compatibility decision; do not "fix" it to bcrypt/scrypt without a migration plan.
- Testing switches: `AUTH_DISABLED=true` (full dev bypass, any OTP/MPIN accepted) and `AUTH_STATIC_LOGIN=true` (login returns a pinned real AIBRAHIM39 payload and embeds the full user data in the JWT as a `userdata` claim).

8.2 profile — ops 2, 48, 63

Location: `src/modules/profile` · **Oracle:** `PERSONAL_DETAILS_V`, `EMP_PHONE_V`, `EMP_IN/OUT_ADDRESS_V`, `EMP_CONTACT_V`, `DEP_PHONE_V`, `DEP_ADDRESS_V`, `UPD_PERSONAL_INFO_PR`, `EMP_MARITAL_LOV`

Method	Endpoint	Description	Auth
GET	<code>/profile?enum=&lang=</code>	op 2 — full personal detail (5 views + dependent child rows in parallel)	JWT
POST	<code>/profile/personal</code>	op 48 — update personal details	JWT
GET	<code>/profile/lov/marital-status</code>	op 63 — marital status LOV	JWT

Special: `addressType` / `addressTypeAr` are intentionally NOT collapsed by localization (client request) — see `PRESERVED_TWIN_BASES` in `shared/utils/localize.util.ts`.

8.3 employee — ops 3, 7, 8, 35, 36

Location: `src/modules/employee` · **Oracle:** `EMPLOYMENT_DETAILS_V`, `SALARY_V`, `EMPLOYMENT_V`, `PERFORMANCE_V`, `SUPERVISOR_VIEW` (table function), `SUPERVISOR_PR`

Method	Endpoint	Description	Auth
GET	<code>/employee/employment</code>	op 3 — employment details + salary/assignment history	JWT
GET	<code>/employee/basic</code>	op 8 — basic info	JWT
GET	<code>/employee/performance</code>	op 7 — performance records	JWT
GET	<code>/employee/supervisor/views</code>	op 35 — supervisor candidates (returns <code>PERSON_ID</code> !)	JWT
POST	<code>/employee/supervisor</code>	op 36 — change supervisor (<code>p_new_supervisor = PERSON_ID</code> , not employee number)	JWT

8.4 payslip — ops 5, 6, 11

Location: `src/modules/payslip` · **Oracle:** `GET_PAYSIP_PERIODS`, `CHK_PAYROLL_CNT`, `PAYSIP_PR`

Method	Endpoint	Description	Auth
GET	<code>/payslip/periods</code>	op 5 — pay periods	JWT

Method	Endpoint	Description	Auth
GET	/payslip/count	op 6 — payslip count for a period	JWT
GET	/payslip	op 11 — full payslip (PAYSLIP_PR returns 7 REF CURSORS — earnings, deductions, housing...)	JWT

8.5 leave — ops 9, 10, 12-14, 45-47, 55-58, 61, 62

Location: `src/modules/leave` · **Oracle:** LEAVE_BALANCE_PR, LEAV_OF_ABSEN_NEW_PR, CALC_LEAV_DUR_PR, HR_LEAV_AMEND_PR, HR_LEAV_CANCEL_PR, RET_FRM_LEAV_PR, ABSENCE_V + ~13 LOV views

Method	Endpoint	Description	Auth
GET	/leave/balance? username=&lang=&effectivedate=	op 9 — leave balance (p_user_name bound to the request value as-is; legacy person_id alias accepted)	JWT
POST	/leave/apply	op 10 — leave submission (LeaveApplyBinds builder, attachments as base64 → BLOB)	JWT
POST	/leave/calculate	op 47 — duration calculation	JWT
POST	/leave/amend · /leave/cancel · /leave/return	ops 57/58/56 — submits	JWT
GET	/leaves?user_name=&leave_type=	leave history (ABSENCE_V)	JWT
GET	/leave/lov/types · reasons · classes · defaults · request-lov · return · return-details · return-related1 · return-related2	ops 12/13/14/45/46/55 + RFL LOVs	JWT
GET	/leave/lov/cancel · /leave/lov/amend (? person_id= or ?username=, optional ? leave_type= filter on the view's NAME column)	ops 61/62 — the EXACT strings the cancel/amend submits accept	JWT

Gotchas pinned in code comments: none of the leave procedures accept `p_language`; op 61/62 `used_value` is the only text the submit procedures accept; `RET_FRM_LEAV_PR.p_leave_details` is compacted to fit VARCHAR2(60).

8.6 letters — ops 16, 17

Location: `src/modules/letters` · **Oracle:** 7 LOV views + HR_EMPLOYMNT_LTR_PR

Method	Endpoint	Description	Auth
GET	/letters/lov	op 16 — all letter-request LOVs (parallel fan-out)	JWT
POST	/letters/apply	op 17 — submit letter request	JWT

8.7 identity — ops 18, 19, 53b, 54, 59, 60

Location: `src/modules/identity` · **Oracle:** QID_DET_V, QID_CHG_PR, COID_REQ_PR, SIT_WORK_LOC_V, SIT_DELEV_LOC_V, SIT_REASON_V

Method	Endpoint	Description	Auth
GET	/identity/qid	op 18 — QID details	JWT
POST	/identity/qid/update	op 19 — QID update	JWT
POST	/identity/idcard/apply	op 54 — company ID request	JWT
GET	/identity/lov/work-location · delivery-location · reason	ops 53b/59/60	JWT

8.8 contact — ops 25, 27-30, 32

Location: `src/modules/contact` · **Oracle:** PHONE_TYPE_V, COUNTRY_LOV, PHONE_PKG.ADD_OR_UPDATE_PHONE, DEL_PHONE_NUMBER_PR, CREATE_ADDRESS_PR, UPD_ADDRESS_PR

Method	Endpoint	Description	Auth
GET	/contact/lov/phone-type · /contact/lov/country	ops 27/30	JWT
POST	/contact/phone	op 28 — upsert phones (submitted per-phone; stops at first failure)	JWT
POST	/contact/phone/delete	op 32	JWT
POST	/contact/address · /contact/address/update	ops 29/25 — p_country = country NAME ("Qatar"); date-tracked (same-day repeat update fails)	JWT

8.9 dependents — ops 24, 31, 33, 34, 49, 64, 65

Location: `src/modules/dependents` · **Oracle:** ADD_DEPENDENT_PKG (add/update), REMOVE_DEPENDENT_PR, PASS_DTL_PR, DEP_LOOKUP_LOV, DEP_PLACE_LOV, PASSPORT_TYPE

Method	Endpoint	Description	Auth
POST	/dependents	op 65 — add dependent (strict flexfield rules: ≥1 attachment, passport fields, unique QID, relationship from op-64 CONTACT group)	JWT
POST	/dependents/update · /dependents/delete	ops 24/31	JWT
GET	/dependents/lov	op 64 — multi-type LOV (also serves ADDRESS_TYPE/SPONSORSHIP/VISA groups)	JWT

Method	Endpoint	Description	Auth
GET	/dependents/passport/types · /dependents/passport/issue-place	ops 33/49	JWT
POST	/dependents/passport/apply	op 34	JWT

Accepts the legacy misspelled `p_*` names (`p_gendar`, `p_relation_ship`, `p_visa_validy`, ...) and mirrors them to canonical ones.

8.10 school-fees — ops 37-40, 50, 52, 53

Location: `src/modules/school-fees` · **Oracle:** SCHOOL_FEE_PR, CHILD_DETETS_VIEW (table function), SCHOOL_NAME_LOV (+4 more LOVs)

Method	Endpoint	Description	Auth
POST	/school-fees/apply	op 39 — school-fee request	JWT
GET	/school-fees/lov/schools (search/page/pageSize) · terms · edu-stage · academic-year · request-type	ops 37/38/40/50/53	JWT
GET	/school-fees/children	op 52 — child details	JWT

8.11 annual-ticket — ops 66, 67, 72

Location: `src/modules/annual-ticket` · **Oracle:** ANNUAL_TICKET_LOV, TICKET_REQ_PR, CANCEL_TKT_PR + 3 cancellation views

Method	Endpoint	Description	Auth
GET	/annual-ticket/master	op 66	JWT
POST	/annual-ticket/apply	op 67 — <code>p_employee</code> MUST be the Oracle <code>PERSON_ID</code> ; <code>p_contractual_year</code> must exist in <code>HMC_HR_CONTRACTUAL_YEAR_SIT</code>	JWT
GET	/annual-ticket/cancel-options	op 72 — tickets + takenAs + repayment methods (parallel)	JWT
POST	/annual-ticket/cancel	op 72	JWT

8.12 approvals — ops 20-23, 68-71 + RFMI

Location: `src/modules/approvals` · **Oracle:** APPROVE_SUMRY_V, MY_REQUEST_SUMMARY_V, PNDNG_QID_V, WORKLISTS_V, ACTION_HISTORY_V, NOTYFY_APPR_V, HR_ATTACHMENTS_V, ~10 per-service PNDNG_* detail views (allow-listed), APPROVE_REJECT_PR, HR_RFMI_PR, REASSIGN_PR

Method	Endpoint	Description	Auth
GET	/approvals	op 20 — approvals summary	JWT
GET	/approvals/my-requests	op 23	JWT
GET	/approvals/worklist · /approvals/worklist/summary · /approvals/worklist/{id}/history	ops 68/69/70 (keyed by USERNAME, not employee number)	JWT
GET	/approvals/{id}/details · /approvals/attachments/{documentId}	op 21 + attachment download	JWT
POST	/approvals/{id}/decision · /approvals/{id}/request-info · /approvals/{id}/reassign	ops 22 / RFMI / 71	JWT

8.13 lookups (shared kernel) — ops 15, 26 + generic reads

Location: `src/lookups`

Method	Endpoint	Description	Auth
GET	/lookups/yes-no · /lookups/rfmi-user	ops 15/26	JWT
GET	/lookups/lov?lovname= · /lookups/master? lookupname=	generic reads via the registry in <code>shared/constants/lov-names.ts</code>	JWT

All feature-module LOV endpoints internally call `LookupsService.getByObject(...)` — LOV reads are cached per (object, lang, username, options) for `LOV_CACHE_TTL_MS` (default 5 min) with concurrent-read coalescing.

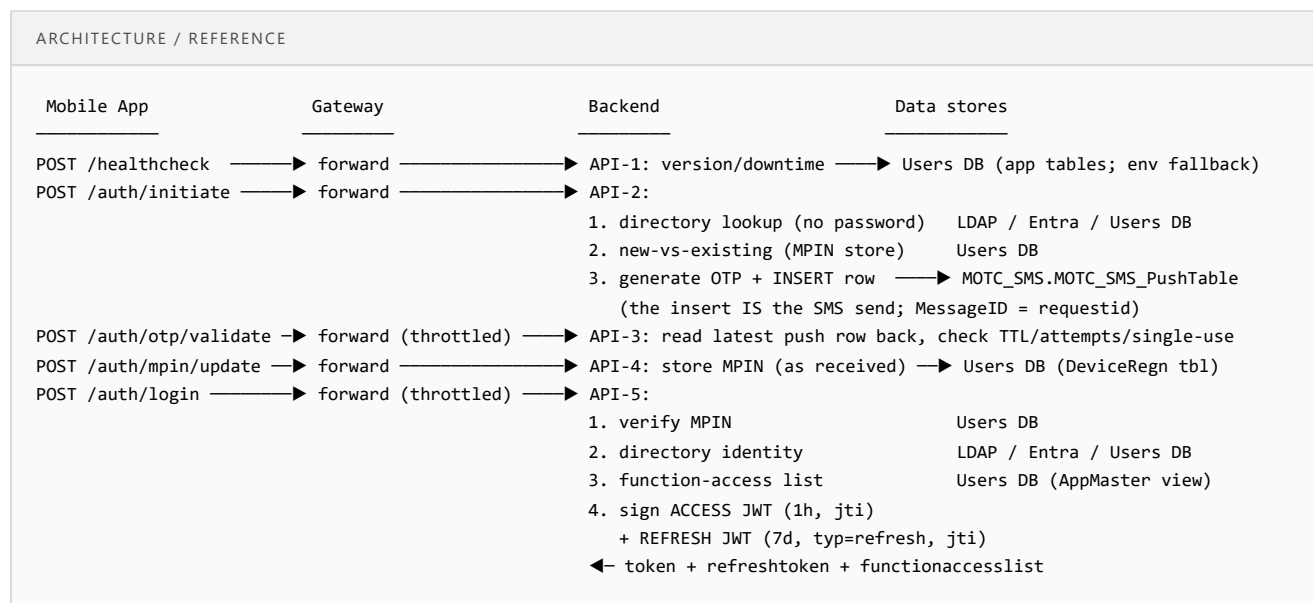
8.14 Gateway routes

Method	Endpoint	Behavior	Auth
POST	/healthcheck, /auth/initiate, /auth/mpin/update	forwarded verbatim	Public
POST	/auth/otp/validate, /auth/login, /auth/mpin/forgot, /auth/mpin/update/reset, /auth/token/refresh	forwarded verbatim + throttled (default 5/min per IP)	Public
ANY	everything else (@A11(' * '))	JWT validated locally → proxied byte-for-byte	JWT

CHAPTER 09 / 21

authentication & authorization

9.1 The login journey (what the mobile app does)



9.2 After login

- The app sends `Authorization: Bearer <access token>` on every call.
- The **gateway** verifies the signature **locally** (shared `JWT_SECRET` — same value in both `.env`s, must stay in sync) and proxies the request. It never calls the backend to validate.
- The **backend** verifies again (`JwtStrategy`), additionally rejecting: refresh tokens used as access tokens (`typ=refresh`), and tokens whose `jti` was revoked by logout (in-memory `TokenRevocationService`). **The backend is the enforcement point for logout.**
- `POST /auth/token/refresh` (public, throttled) exchanges a valid refresh token for a fresh pair; the used refresh token is revoked (one-time use / rotation).
- `POST /auth/logout` revokes the current access token's `jti` (+ the refresh token if sent in the body).

JWT claims: `sub` (employee number or username), `username`, `employeeNumber`, `roles` (e.g. `["EMPLOYEE"]`), `functions` (enabled function codes), `name`, `dept`, `company`, `jti`, `iat`, `exp` — plus a full `userdata` object when `AUTH_STATIC_LOGIN=true`.

9.3 Authorization

- **RolesGuard** — routes decorated with `@Roles(Role.APPROVER, ...)` require that role in the token (approvals module).
- **FunctionAccessGuard** — routes decorated with `@RequireFunction('frmSchoolFees')` require that function code in the token's `functions` claim (which came from the Users-DB AppMaster view at login).
- Undecorated routes need only a valid token.

9.4 Switchable providers (all decided by env, no redeploy)

Concern	Env switch	Options
Employee directory	AUTH_DIRECTORY	ldap (LDAPS/Idapts) · entra (Microsoft Graph app-only) · usersdb (legacy device-table only, no corporate directory)
OTP store + delivery	OTP_STORE	motc (default, MOTC_SMS_PushTable) · legacy (HMC_RHAP_OTP_tbl + HTTP SMS adapter)
Everything (dev)	AUTH_DISABLED=true	full bypass: guards permissive, synthesized identities, any well-formed OTP/MPIN accepted
Static login (testing)	AUTH_STATIC_LOGIN=true	login returns a pinned real payload; full user data embedded in the JWT; rest of API enforces tokens normally

9.5 Security invariants (do not break these)

- MPIN values are stored as received (legacy decision — see §21 before changing).
- SQL is always parameterized (`:binds` in Oracle, `@params` in SQL Server). Object/table names interpolated into SQL are validated against allow-lists (`ORACLE_OBJECTS` , identifier regexes).
- Known limitation: token revocation is in-memory — a backend restart forgets logouts (until token expiry), and multiple instances don't share the denylist. Accepted trade-off; documented in `TokenRevocationService` .

CHAPTER 10 / 21

database architecture

There is no ORM, no entities, no migrations — on purpose. All schemas are owned by other teams (Oracle HR, legacy Sanaad SQL Server, MOTC). This project only *reads views* and *calls procedures*, with parameterized SQL.

10.1 The three databases

Database	Driver/Pool	What lives there	Service
Oracle HR (XXHMC_SND_* schema)	node-oracledb pool	~95 allow-listed views/LOVs/procedures implementing legacy business operations	core/database/oracle.service.ts
Users/Sanaad DB (SQL Server)	mssql pool #1 (USERS_DB_*)	HMC_Sanad_DeviceRegn_tbl (device+MPIN), HMC_RHAP_OTP_tbl (legacy OTP), app master/downtime/update tables, HMC_Sanad_AppMaster_VW (function access)	core/database/mssql.service.ts
MOTC_SMS DB (SQL Server, named instance, static port 9001)	mssql pool #2 (MOTC_SMS_DB_*)	MOTC_SMS_PushTable — the government SMS outbox; OTP rows are stored AND delivered through it	core/database/motc-sms-db.service.ts

10.2 How Oracle access works (the most important pattern in the codebase)

Problem this project solved: the legacy documentation describes *request parameters*, not the actual database objects — column names and procedure signatures differ per object. Guessing produced `ORA-00904`, `PLS-00306`, etc.

Solution — **ask the database itself** (`OracleSchemaService` reads the data dictionary) and centralize the calling patterns in `BaseOracleRepository`, which every Oracle adapter extends:

Helper	Use case	Behavior
<code>query(sql, binds)</code>	plain view SELECT	parameterized
<code>readByResolvedKey(view, value, candidates)</code>	"filter this view by user" when the key column name varies (USER_NAME vs USERNAME vs EMPLOYEE_NUMBER...)	resolves the real column from ALL_TAB_COLUMNS; on schema mismatch returns [] instead of failing the request

Helper	Use case	Behavior
readByResolvedKeyAny / In	same, matching several identifier forms / IN-lists	chunked, parameterized
callSubmitProc(proc, params, values)	business submits (*_PR)	resolves the DECLARED argument list (incl. overloads) from ALL_ARGUMENTS, binds by name, maps OUT params (p_success_flag/p_error_msg[_ar]) to a SubmitResult
callRowsProc(proc, params, values)	procedures returning a REF CURSOR	finds the cursor parameter's real name
callMultiCursorProc	PAYSLIP_PR's 7 cursors	reads all cursors on one connection
callRowsOrTableFunction	object may be a proc OR a table function	dictionary decides; SELECT * FROM TABLE(fn(...)) fallback

Allow-list rule: any Oracle object name must exist in `shared/constants/oracle-objects.ts` before it can be queried — this is both an injection defense and a catalog.

10.3 SQL Server access

Both `MssqlService` and `MotcSmsDbService` expose parameterized `query / execute` (named `@params`). Pools are created at boot; `*_DISABLED=true` skips creation (routes then fail with a typed 503 while the rest of the API works).

10.4 Data flow summary

ARCHITECTURE / REFERENCE	
Read op:	Controller → Service → OracleRepository.readByResolvedKey(VIEW) → rows → mapper (col/str/dateStr, URL-decode Arabic) → ResponseInterceptor → localizeArTwins (EN/AR twin collapsing per ?lang=) → Sanaad envelope → client
Submit op:	Controller (strict DTO) → Service → OracleRepository.callSubmitProc(PROC) → OUT params → SubmitResult {successflag, message, messageAr} → HTTP 200 ALWAYS (Sanaad convention: business result is successflag, not the status code)

CHAPTER 11 / 21

external integrations

11.1 Oracle HR (business core) — see §10.2.

11.2 Users/Sanaad SQL Server — auth cycle + API-1

Config: `USERS_DB_*`. Implemented in `modules/auth/infrastructure/adapters/mssql-*.ts`. Failure mode: typed 503 per call; healthcheck falls back to `APP_*` env config.

11.3 MOTC SMS gateway DB — OTP store + SMS delivery

Config: `MOTC_SMS_DB_*` + fixed INSERT column values (`MOTC_SMS_APP_ID`, `MOTC_SMS_SUBJECT_ID`, priority/language/etc. — several still pending from the client). Implemented in `modules/auth/infrastructure/adapters/motc-sms-otp.repository.ts` (+ `core/database/motc-sms-db.service.ts`). `MessageID` is generated as MAX+1 with duplicate-key retry. `BusinessParam1/2` carry username+IMEI by default (what makes per-user OTP validation possible). The OTP text template is `SMS_MESSAGE_TEMPLATE` (`{otp}` placeholder).

11.4 LDAP / Active Directory (LDAPS)

Config: `LDAP_*` (host, base DN, bind account, CA cert inline or path). Implemented in `ldap-user.repository.ts` (ldaps). Only `validate()` (passwordless identity+phone lookup) is used by the journey. TLS: provide `LDAP_CA_CERT[_PATH]` and set `LDAP_TLS_REJECT_UNAUTHORIZED=true` in production.

11.5 Microsoft Entra ID (Graph) — alternative directory

Config: `ENTRA_*` (tenant, client id/secret — app needs `User.Read.All` application permission with admin consent). Implemented in `entra-graph-user.repository.ts` (client-credentials token + Graph user lookup). Selected with `AUTH_DIRECTORY=entra`.

11.6 Legacy HTTP SMS gateway (only when `OTP_STORE=legacy`)

Config: `SMS_API_*`. Implemented in `sms-otp-delivery.adapter.ts` — a generic JSON POST placeholder awaiting the corporate contract. Unset base URL = hard 503 in production.

Error-handling rule for ALL integrations: adapter throws a typed exception → `AllExceptionsFilter` classifies it → client gets a safe, generic message.

CHAPTER 12 / 21

configuration & environment variables

Both projects: `.env` (never committed) → Joi schema validates at boot (bad env = refusal to start) → typed namespaces in `core/config/configuration.ts` → injected via `ConfigService`.

12.1 Backend (HMC_BackEnd) — grouped; placeholders only

Variable	Required	Purpose	Example
NODE_ENV	no (default development)	environment mode; production hardens several features	production
PORT / API_PREFIX / CORS_ORIGINS	no	listener + prefix + CORS	3009 / api/v1 / *
REQUEST_TIMEOUT_MS	no	global request timeout (408)	30000
Oracle ORACLE_USER / ORACLE_PASSWORD / ORACLE_DSN	yes (unless disabled)	pool credentials + EZConnect DSN	xxhmc_snd / YOUR_PASSWORD / host:1521/SERVICE
ORACLE_POOL_MIN/MAX, ORACLE_QUEUE_TIMEOUT_MS, ORACLE_CALL_TIMEOUT_MS	no	pool + per-call timeouts	2/10/25000/25000
ORACLE_DISABLED / ORACLE_THICK_MODE / ORACLE_CLIENT_LIB_DIR	no	skip pool · thick mode + Instant Client path	false/true/`
Users DB USERS_DB_HOST/PORT/NAME/USER/PASSWORD	yes (unless disabled)	SQL Server #1	sqlhost.hmc.org.qa/1433/.../YOUR_PASSWORD
USERS_DB_ENCRYPT / USERS_DB_TRUST_SERVER_CERT / USERS_DB_DISABLED	no	TLS · self-signed trust · skip pool	true/false/false
MOTC SMS DB MOTC_SMS_DB_HOST/PORT/NAME/USER/PASSWORD	yes for OTP (unless disabled)	SQL Server #2 (named instance + static port)	HSCL7VVSQ1\SQL1 / 9001 / MOTC_SMS / Sanaad_Liv_Usr / YOUR_PASSWORD
MOTC_SMS_DB_ENCRYPT / MOTC_SMS_DB_TRUST_SERVER_CERT / MOTC_SMS_DB_DISABLED	no	TLS · trust self-signed · skip pool	true/true/false

Variable	Required	Purpose	Example
MOTC_SMS_TABLE, MOTC_SMS_APP_ID, MOTC_SMS_FROM_ADDRESS, MOTC_SMS_SUBJECT_ID, MOTC_SMS_PRIORITY, MOTC_SMS_LANGUAGE_ID, MOTC_SMS_RECIPIENT_ADDRESS_TYPE, MOTC_SMS_PROCESSED_STATE, MOTC_SMS_MESSAGE_EXPIRE_MINUTES, MOTC_SMS_CUSTOMER_ID, MOTC_SMS_BUSINESS_PARAM1/2	APP_ID+SUBJECT pending from client	fixed columns of the documented INSERT	see .env.example
OTP_STORE	no	motc (default) or legacy	motc
OTP_LENGTH / OTP_TTL_SECONDS / OTP_MAX_ATTEMPTS / OTP_RESEND_WINDOW_SECONDS	no	OTP policy	6/300/5/60
SMS (legacy) SMS_API_BASE_URL/KEY, SMS_SENDER_ID, SMS_MESSAGE_TEMPLATE	only for legacy store	HTTP SMS adapter · OTP text ({otp})	https://.../YOUR_KEY/HMC/Your Sanaad verification code is {otp}
JWT JWT_SECRET / JWT_ISSUER / JWT_AUDIENCE / JWT_EXPIRES_IN / JWT_REFRESH_EXPIRES_IN	SECRET: yes	token signing — must match the gateway	YOUR_LONG_RANDOM_SECRET/sanaad/sanaad-b2e/1h/7d
AUTH_DISABLED / AUTH_STATIC_LOGIN	no (never true in prod)	dev bypass · static test login	false/false
AUTH_DIRECTORY	no	ldap · entra · usersdb	usersdb
FUNCTION_ACCESS_VIEW / FUNCTION_ACCESS_APP_ID	no	login function list source	HMC_Sanad_AppMaster_Vw/1
MPIN MPIN_MIN/MAX_LENGTH, MPIN_MAX_ATTEMPTS, MPIN_LOCKOUT_MINUTES	no	MPIN policy	4/6/5/15
LDAP LDAP_ENABLED, LDAP_HOST/PORT, LDAP_BASE_DN, LDAP_SEARCH_FILTER, LDAP_BIND_DN/PASSWORD, LDAP_CA_CERT[_PATH], LDAP_TLS_REJECT_UNAUTHORIZED, LDAP_TIMEOUT_MS	when AUTH_DIRECTORY=ldap	directory lookup	see .env.example
Entra ENTRA_TENANT_ID / ENTRA_CLIENT_ID / ENTRA_CLIENT_SECRET (+ URLs, lookup attr)	when AUTH_DIRECTORY=entra	Graph lookup	YOUR_TENANT/YOUR_CLIENT_ID/YOUR_SECRET

Variable	Required	Purpose	Example
App launch APP_NAME, APP_MIN_SUPPORTED_VERSION, APP_LATEST_VERSION, APP_DOWNTIME(_START/_END)	no	API-1 fallback config	SanaadHealth...
LOV_CACHE_TTL_MS	no	LOV cache lifetime	300000

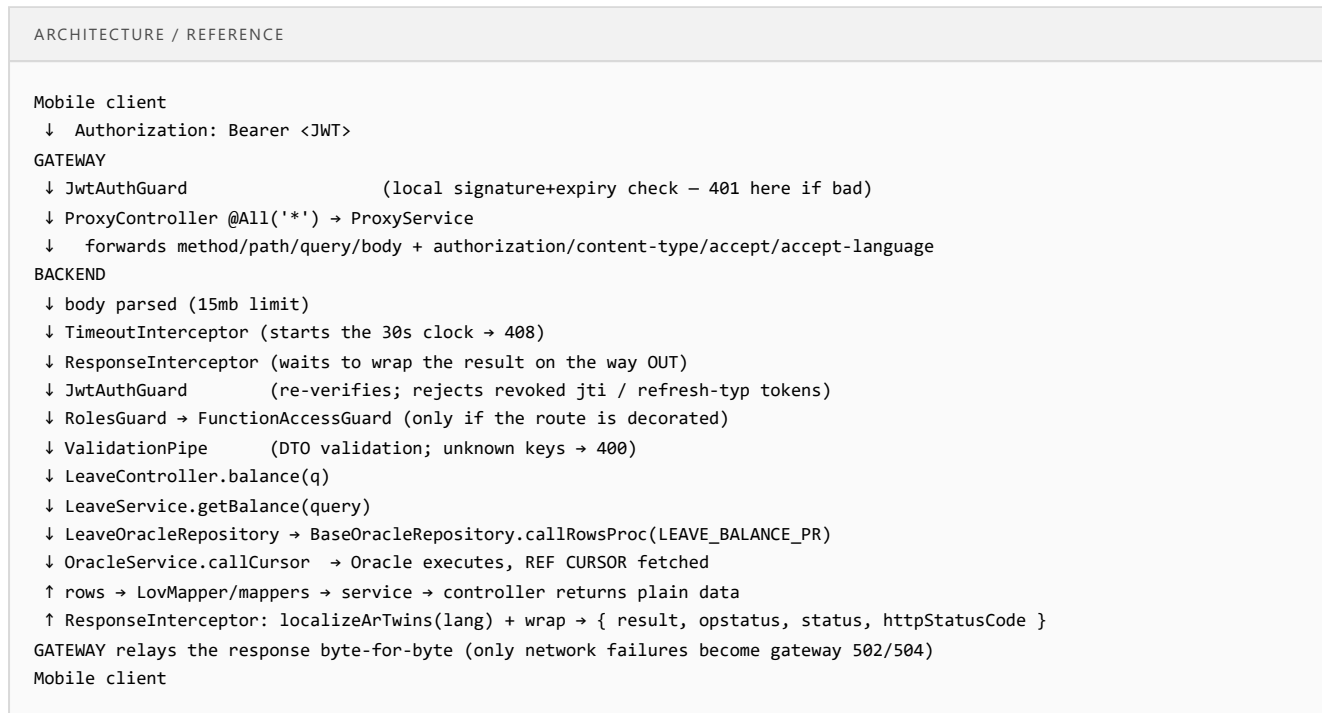
12.2 Gateway (HMC_Gateway)

Variable	Required	Purpose	Example
GATEWAY_PORT / API_PREFIX / CORS_ORIGINS	no	listener	3001/api/v1/*
BACKEND_BASE_URL / BACKEND_API_PREFIX	yes	upstream backend	http://localhost:3009 / api/v1
BACKEND_TIMEOUT_MS	no	proxy timeout — keep it ABOVE the backend's REQUEST_TIMEOUT_MS (e.g. 35000 vs 30000) so backend errors win over generic 504s	35000
BACKEND_CA_CERT / BACKEND_CA_CERT_PATH / BACKEND_TLS_REJECT_UNAUTHORIZED	for https backend	TLS trust	// true
JWT_SECRET / JWT_ISSUER / JWT_AUDIENCE	yes	must equal the backend's values	YOUR_LONG_RANDOM_SECR ET
AUTH_DISABLED	no (never true in prod)	bypass gateway JWT check	false
THROTTLE_LOGIN_LIMIT / THROTTLE_LOGIN_TTL_MS	no	rate limit for login surfaces	5/60000
REQUEST_TIMEOUT_MS / NODE_ENV	no	misc	35000/development

CHAPTER 13 / 21

api request lifecycle

An authenticated business request (GET /api/v1/leave/balance) end to end:



Error path: ANY exception thrown anywhere is caught by `AllExceptionsFilter`, classified, and returned as a safe envelope: `{ success:false, message, status:'error', httpStatusCode }`.

CHAPTER 14 / 21

error handling

14.1 The error pipeline

1. **Validation errors** — global `ValidationPipe` rejects malformed/unknown fields → 400 with detail list.
2. **Typed domain errors** — adapters throw meaningful exceptions (`OracleQueryError` , `MssqlQueryError` , `MssqlUnavailableException` , `NotFoundException` , ...).
3. **AllExceptionsFilter** (backend) catches everything, uses the **exception classifier** (`core/http/exception-classifier.ts`) to bucket it (`VALIDATION` / `DATABASE_ERROR` / `EXTERNAL_SERVICE` / `AUTH` / `TIMEOUT` / `UNKNOWN`), picks a safe bilingual message (`shared/constants/error-codes.ts` maps ORA codes → HTTP statuses), and returns the consistent error envelope.
4. **Sanaad submit convention** — business submits return **HTTP 200 even on business failure**; the outcome is in `successflag` (`S` / `E`) + `message` / `messageAr` . Don't "fix" this — the mobile app depends on it.
5. Gateway: relays backend errors untouched; only a real network failure produces a gateway-originated `502` (unreachable) or `504` (timeout).

CHAPTER 15 / 21

how to develop a new feature

Follow the existing architecture — copy the closest existing module rather than inventing a new style. `identity/` or `letters/` are good small templates.

15.1 Add a new module

ARCHITECTURE / REFERENCE

```
src/modules/<name>/
├─ <name>.module.ts
├─ interface/<name>.controller.ts + interface/dto/<name>.dto.ts
├─ application/<name>.service.ts
├─ domain/<name>.repository.ts      ← interface + `export const X_REPOSITORY = Symbol(...)`
└─ infrastructure/oracle/<name>.oracle.repository.ts ← extends BaseOracleRepository
```

1. Declare the module: controller in `controllers`, service + `{ provide: X_REPOSITORY, useClass: XOracleRepository }` in `providers`.
2. Import it in `src/app.module.ts`.
3. If it reads any new Oracle object: **add the name to** `shared/constants/oracle-objects.ts` **first** (queries against unknown objects are rejected).

15.2 Add a new endpoint to an existing module

1. Add a DTO in `interface/dto/` (extend `LangQueryDto` / `ProfileQueryDto` for reads; use the helpers in `shared/dto/oracle-submit.dto.ts` for `p_*` submit bodies).
2. Add the controller method with `@Get/@Post`.
3. Add the service method; call the repository port — never SQL in the service.
4. Implement the adapter method with the right base helper (`readByResolvedKey` for view reads, `callSubmitProc` for submits, `callRowsProc` for REF-CURSOR procs).
5. Add a `*.spec.ts` beside the adapter mocking `OracleService` / `OracleSchemaService` (copy an existing spec's `make()` pattern).
6. `npm run build + npm test`, then verify by hand in Postman.

15.3 Add validation

Use class-validator decorators on the DTO — the global pipe does the rest. Remember `forbidNonWhitelisted: true`: every accepted field must be declared. Legacy alternate spellings are handled by declaring both and mirroring in the adapter (see `dependents.binds.ts`).

15.4 Database changes

There are no migrations here — schema changes happen on the Oracle/SQL Server side by their owners. When a procedure/view changes:

1. Confirm the updated columns or procedure signature with the database owners.
2. Usually nothing else is needed — signatures are resolved at runtime. Update the documented param list / DTO only if the *request contract* changed.
3. Record any live-verified behavior in `AGENTS.md` (that file is the team's memory).

15.5 Protect an endpoint

- Default: authenticated by the global guard (do nothing).
- Public: `@Public()` (only for the pre-login journey!).
- Role-restricted: `@Roles(Role.APPROVER)`.
- Function-restricted: `@RequireFunction('frmSchoolFees')`.
- If you add a truly new public route, remember the gateway: it must either be declared there (like the auth journey) or it will require a JWT at the gateway.

CHAPTER 16 / 21

testing & debugging

16.1 What exists

- **Backend:** 22 unit suites / ~170 tests, co-located as `src/**/*.spec.ts` (Oracle adapters with mocked pools, OTP stores, auth service, guards, utils). **No e2e specs** (`npm run test:e2e` runs nothing — the config exists, the specs were never written).
- **Gateway:** 4 unit suites + a real e2e suite (`test/gateway.e2e-spec.ts`) that boots the app against `test/mock-backend.ts` (mints real JWTs) — covers proxying, auth, throttling, 502 mapping.

16.2 Running tests

BASH / REFERENCE

```
npm test          # all unit tests
npm test -- --watch # watch mode
npm run test:cov   # coverage report → coverage/
npx jest src/modules/leave # one folder/file
npm run test:e2e   # gateway only (backend has no e2e specs)
```

Tests never need a database — everything external is mocked.

16.3 Debugging in VS Code

1. Run `npm run start:debug` (starts `ts-node-dev` with `--inspect`).
2. VS Code → Run and Debug → "Attach to Node Process" (or add a launch config with `"request": "attach", "port": 9229`).
3. Set breakpoints directly in the `.ts` files (source maps work), e.g. in a controller method.
4. Trigger the request from Postman (`postman/HMC-Sanaad-Full.postman_collection.json` has verified examples), using a bearer token from `/auth/login` for protected endpoints.

Debugging a single Jest test: `npx jest path/to.spec.ts --runInBand` with a `debugger;` statement, launched from VS Code's JavaScript Debug Terminal.

CHAPTER 17 / 21

important npm commands

Same scripts in both projects (run with `npm run <name>`):

Command	Purpose
build	Compile TS → dist/ (nest build + tsc-alias for path aliases). Also the typecheck.
start / start:prod	Run the compiled app (node dist/main)
start:dev	Development server with hot reload (ts-node-dev)
start:debug	Dev server with the Node inspector for VS Code attach
test / test:watch / test:cov	Jest unit tests / watch / coverage
test:e2e	e2e tests (gateway has them; backend currently has none)
lint	ESLint with --fix Caution: on the backend this crashes (known env issue) — use npx eslint src --ext .ts instead, and ignore the CRLF Delete CR noise
format	Prettier Caution: do NOT run on the backend (CRLF working copy — it would rewrite every file). Gateway is fine.
depccruise (backend only)	dependency-rule check (.dependency-cruiser.js)

CHAPTER 18 / 21

troubleshooting

Symptom	Likely cause → fix
App refuses to start with a Joi message	invalid/missing env var — the message names it; fix <code>.env</code>
Boot fails: Failed to create <X> DB pool	that DB is unreachable/misconfigured. Boot intentionally fails. Temporarily set <code>ORACLE_DISABLED / USERS_DB_DISABLED / MOTC_SMS_DB_DISABLED =true</code> to boot without it
...self-signed certificate (SQL Server)	set <code>USERS_DB_TRUST_SERVER_CERT=true / MOTC_SMS_DB_TRUST_SERVER_CERT=true</code> (internal instances use self-signed certs)
Login failed for user '...' (SQL Server)	credential/permission issue — verify password (quote values containing # in <code>.env</code>), then ask the DB team for the 18456 state code (5=no login, 8=bad password, 38=no DB access, 58=Windows-auth-only)
<code>getaddrinfo ENOTFOUND <shortname></code> in Docker	Linux containers can't resolve short Windows hostnames → use FQDN/IP or <code>extra_hosts</code>
<code>EADDRINUSE</code> port already in use	another instance running → change <code>PORT/GATEWAY_PORT</code> or kill the process (<code>netstat -ano</code>)
Gateway returns <code>{"statusCode":504}</code>	backend didn't answer within <code>BACKEND_TIMEOUT_MS</code> — check backend boot state and which dependency hangs
401 on everything	missing/expired/revoked token, or <code>JWT_SECRET</code> mismatch between gateway and backend (they MUST be identical)
OTP endpoints return 503	MOTC pool not created or <code>OTP_STORE=legacy</code> without SMS config
Oracle <code>MODULE_NOT_FOUND</code> from dist/	stale incremental build artifact → delete <code>tsconfig.build.tsbuildinfo</code> , rebuild (and never re-add <code>"incremental": true</code>)
ESLint <code>TypeError: expand is not a function</code>	known env issue → <code>npx eslint src --ext .ts</code>
Thousands of <code>Delete CR</code> lint errors	CRLF working copy — filter them out; do not run <code>prettier --fix</code> on the backend
400 property <code>X</code> should not exist	<code>forbidNonWhitelisted</code> — add the field to the DTO (or fix the client payload)
413 / body too large	body limit is 15mb (<code>BODY_LIMIT</code> in <code>main.ts</code>) — attachments are base64 (~4/3 inflation)

CHAPTER 19 / 21

learning roadmap (first week)

Day 1 — run it. Install Node 20 + VS Code. Clone. `npm ci` in both projects. Do the §3.5 no-database smoke test, then send the §3.6 `/auth/login` request in Postman. Read `AGENTS.md` top to bottom (skim is fine).

Day 2 — language basics + bootstrap. Work through §4 with the files open: `package.json`, `HMC_BackEnd/src/main.ts`, `core/config/configuration.ts` + `env.validation.ts`. Change a default (e.g. PORT), watch Joi reject a bad value.

Day 3 — one full request. With `start:debug` running and §13 next to you, follow `GET /api/v1/lookups/yes-no?lang=en` from Postman through `lookups.controller.ts` → `lookups.service.ts` → `lov.oracle.repository.ts` with breakpoints (see §16.3). Then read `core/core.module.ts` (pipeline order) and `core/http/response.interceptor.ts`.

Day 4 — auth + data layer. Read §9, then `modules/auth/application/auth.service.ts`, `core/auth/jwt.strategy.ts`, and the port/adapters in `modules/auth`. Then §10 + `core/database/base.repository.ts` (the helpers) and one adapter (`leave.oracle.repository.ts`). Log in via Postman (`AUTH_DISABLED=true`), decode your JWT at jwt.io.

Day 5 — make a change. Pick something small: add a query filter to an existing LOV endpoint (mirror §15.2), write its spec, run build+tests, verify in Postman. Then debug one existing test in VS Code. Bonus: run the gateway e2e suite and read `gateway.e2e-spec.ts`.

CHAPTER 20 / 21

learning resources

Node.js / npm

- Official docs: <https://nodejs.org/docs/latest-v20.x/api/>
- npm basics: <https://docs.npmjs.com/getting-started>

TypeScript

- Handbook (start with "TypeScript for JS Programmers"):
<https://www.typescriptlang.org/docs/handbook/intro.html>

NestJS (the most important one)

- Official docs — read First Steps → Controllers → Providers → Modules → Guards → Interceptors → Pipes → Exception filters: <https://docs.nestjs.com>
- Config: <https://docs.nestjs.com/techniques/configuration> · Validation: <https://docs.nestjs.com/techniques/validation>
- Passport/JWT: <https://docs.nestjs.com/recipes/passport>
- Testing: <https://docs.nestjs.com/fundamentals/testing>

Databases

- node-oracledb docs: <https://node-oracledb.readthedocs.io/en/latest/>
- mssql (tedious) docs: <https://www.npmjs.com/package/mssql>
- Oracle data dictionary (ALL_TAB_COLUMNS/ALL_ARGUMENTS) reference: <https://docs.oracle.com/en/database/oracle/oracle-database/19/refrn/>

Auth

- JWT introduction: <https://jwt.io/introduction>
- passport-jwt: <https://www.passportjs.org/packages/passport-jwt/>

Tooling

- Jest: <https://jestjs.io/docs/getting-started> · Supertest: <https://www.npmjs.com/package/supertest>
- Docker: <https://docs.docker.com/get-started/>
- VS Code Node debugging: <https://code.visualstudio.com/docs/nodejs/nodejs-debugging>

Video (well-maintained, beginner-friendly)

- NestJS official course hub: <https://courses.nestjs.com/> (paid, by the framework authors)
- freeCodeCamp "NestJS Course for Beginners" (YouTube, free): https://www.youtube.com/watch?v=GHTA143_b-s

CHAPTER 21 / 21

requires knowledge transfer

Confirmed from source code (you do NOT need anyone for these)

Architecture, the covered endpoints and their Oracle/SQL mappings, the auth journey, configuration surface, error handling, testing, local/no-DB development, Docker build — documented above and verifiable in the repo. A large amount of *live-verified* business behavior (flexfield rules for op 65, PERSON_ID requirements for ops 36/67, date-track rules, staging DB defects) is preserved in `AGENTS.md`, DTO doc-comments, and the Postman collection's captured real responses.

Requires team knowledge — ask for these explicitly

#	Item	Why it matters	Who probably knows
1	Production/staging infrastructure: servers/jumpstations, how deployments are performed (no CI/CD exists in the repos), who operates Docker hosts	you cannot ship without it	previous developer / DevOps
2	All real credentials: Oracle, Users DB, MOTC SMS DB (password was shared via SMS), LDAP bind account, Entra client secret, production JWT_SECRET	app cannot connect	DevOps / DB team / client
3	MOTC SMS fixed INSERT values: MOTC_SMS_APP_ID, MOTC_SMS_SUBJECT_ID, priority/language/recipient-type codes	OTP rows currently insert NULL/defaults for these; the government gateway may require exact values	client (MOTC integration owner)
4	Corporate HTTP SMS gateway contract (legacy OTP path)	SmsOtpDeliveryAdapter is a placeholder	client
5	Oracle/SQL schema ownership + defect process: staging defects exist in SCHOOL_FEE_PR (ORA-01403), ADD_OR_UPDATE_PHONE (rejects all phone types), UPDATE_DEPENDENT_PR (intermittent ORA-00027) — correct requests fail inside the DB	you'll be blamed for DB bugs otherwise	Oracle apps team
6	Business rules not encoded anywhere: op-51 (skipped — why?), exact role assignment rules (who gets APPROVER), MPIN client-side hashing algorithm	product correctness	client / mobile team
7	Mobile app team contact + release coordination: response-shape changes must be coordinated (e.g. HTTP-200-on-business-failure convention, addressType twin exception)	breaking the app silently	mobile team

#	Item	Why it matters	Who probably knows
8	Security review decisions: MPIN stored as received (legacy equality), in-memory token revocation, trustServerCertificate usage — accepted trade-offs that may need revisiting	compliance	previous developer / security
9	docker-compose env gap: newer vars (MOTC_SMS_*, OTP_STORE, AUTH_STATIC_LOGIN) are not in the compose environment: pass-through — decide whether to add them or manage via .env on servers	deployment surprises	DevOps
10	Git remote / branching / code review process — the repos have no CI config; the workflow is undocumented	collaboration	previous developer

End of handover document. Companion file: `PROJECT_ANALYSIS.md` (raw technical analysis). Keep `AGENTS.md` updated as you learn — it is the living memory of this project.